

Question 1: What is broadcasting in NumPy? State the broadcasting rules (comparing shapes right-to-left) and explain, with a suitable example, how a 1D array can be added to a 2D array without explicitly copying data

Answer :- **Broadcasting** in NumPy is a mechanism that allows arithmetic operations to be performed on arrays of different shapes. Instead of requiring arrays to be identical in shape, NumPy automatically expands the smaller array across the missing or size-1 dimensions so that both arrays have compatible shapes for element-wise operations. This occurs without physically copying the data in memory, making broadcasting memory-efficient and computationally fast.

Broadcasting Rules (Right-to-Left Comparison)

When operating on two arrays, NumPy compares their shapes element-wise **starting from the trailing (rightmost) dimensions** and working its way to the left:

1. **Alignment:** If the two arrays have a different number of dimensions, the shape of the array with fewer dimensions is padded with ones on its **left** side until both shapes have equal lengths.
2. **Compatibility:** Two dimensions are compatible if:
 - They are equal in size, **OR**
 - One of the dimensions is equal to **1**.
3. **Incompatibility:** If neither condition is met at any dimension (i.e., the dimensions are unequal and neither is **1**), NumPy raises a **ValueError: operands could not be broadcast together**.

Example: Adding a 1D Array to a 2D Array

Consider adding a 1D array of shape **(3,)** to a 2D array of shape **(3, 3)**:

Python

```
import numpy as np

# Create a 2D array of shape (3, 3)
A = np.array([[10, 20, 30],
              [40, 50, 60],
              [70, 80, 90]])

# Create a 1D array of shape (3,)
B = np.array([1, 2, 3])

# Add the 1D array to the 2D array
result = A + B

print(result)
```

Output:

Plaintext

```
[[11 22 33]
```

```
 [41 52 63]
```

[71 82 93]]

How Broadcasting Works Step-by-Step:

1. Shape Comparison (Right-to-Left):

- Shape of A: (3, 3)
- Shape of B: (3,) -> Padded on the left to (1, 3)

2. Dimension Matching:

- **Trailing dimension:** Both have size 3 (compatible).
- **Leading dimension:** A has 3, B has 1 (compatible because B's dimension is 1).

3. Execution Without Data Copying:

- NumPy conceptually stretches B across axis 0 from shape (1, 3) to (3, 3) so that [1, 2, 3] is added to every row of A.
- Under the hood, NumPy reuses the single row of B using **stride tricks** (setting the stride for axis 0 to 0), avoiding any actual replication or allocation of duplicate memory.

Question 2: Write a NumPy program that creates a 2D array of shape (3, 4) using np.arange() and reshape(). Compute the sum along axis=0 and axis=1 using np.sum(). Explain, in your own words, what axis=0 and axis=1 mean and why the two results are different.

Answer :- NumPy Program

Python

```
import numpy as np

# Create a 1D array of 12 elements (0 to 11) and reshape it to (3, 4)
arr = np.arange(12).reshape(3, 4)

print("Original 2D Array:")
print(arr)

# Compute sum along axis=0 (columns / down the rows)
sum_axis_0 = np.sum(arr, axis=0)

# Compute sum along axis=1 (rows / across the columns)
sum_axis_1 = np.sum(arr, axis=1)

print("\nSum along axis=0:", sum_axis_0)
print("Sum along axis=1:", sum_axis_1)
```

Output:

Plaintext

Original 2D Array:

```
[[ 0  1  2  3]
```

```
 [ 4  5  6  7]
```

```
 [ 8  9 10 11]]
```

Sum along axis=0: [12 15 18 21]

Sum along axis=1: [6 22 38]

Explanation of **axis=0** vs **axis=1**

In NumPy, an **axis** represents a direction or dimension along which an operation is applied:

1. **axis=0** (Vertical Axis / Column-wise operation):

- **What it means:** Operating along **axis=0** means performing the action **down the rows** (vertically) for each column.
- **Why it collapses:** The operation collapses the vertical dimension (3 rows), leaving behind a 1D array containing the sum of elements in each of the 4 columns.
- **Result:** $[0+4+8, 1+5+9, 2+6+10, 3+7+11] = [12, 15, 18, 21]$ (length 4).

2. **axis=1** (Horizontal Axis / Row-wise operation):

- **What it means:** Operating along **axis=1** means performing the action **across the columns** (horizontally) for each row.

- **Why it collapses:** The operation collapses the horizontal dimension (4 columns), leaving behind a 1D array containing the sum of elements in each of the 3 rows.
- **Result:** $[0+1+2+3, 4+5+6+7, 8+9+10+11] = [6, 22, 38]$ (length 3).

Why the Two Results are Different

- **Direction of Collapse:** `axis=0` sums elements across 3 rows for each column, whereas `axis=1` sums elements across 4 columns for each row.
- **Resulting Dimensions:** The shape of the original array is $(3, 4)$. When performing an operation along an axis, that specific dimension is collapsed:
 - Summing along `axis=0` removes the 1st dimension (size 3) and yields **4 values**.
 - Summing along `axis=1` removes the 2nd dimension (size 4) and yields **3 values**.

Question 3: What is the purpose of `np.random.seed()`? Write a program that uses `np.random.rand()`, `np.random.randn()`, and `np.random.randint()` to generate three different types of random data, and explain the distribution each function draws from.

Purpose of `np.random.seed()`

The purpose of `np.random.seed()` is to **initialize the pseudo-random number generator (PRNG)** in NumPy. Computers generate pseudo-random numbers using deterministic algorithms. Setting a specific seed value ensures that the sequence of generated random numbers is identical every time the code is executed.

Key Benefits:

- **Reproducibility:** Allows scientists, researchers, and developers to reproduce exact results and outputs.
- **Debugging:** Helps in debugging by producing predictable inputs for tests.

Python Program

Python

```
import numpy as np
```

```
# Set the seed for reproducibility
```

```
np.random.seed(42)
```

```
# 1. Uniform distribution using rand()
```

```
data_rand = np.random.rand(5)
```

```
# 2. Standard Normal distribution using randn()
```

```
data_randn = np.random.randn(5)
```

```
# 3. Discrete Uniform distribution using randint()
```

```
data_randint = np.random.randint(low=1, high=100, size=5)
```

```
print("np.random.rand():\n", data_rand)

print("\nnp.random.randn():\n", data_randn)

print("\nnp.random.randint():\n", data_randint)
```

Sample Output:

Plaintext

np.random.rand():

```
[0.37454012 0.95071431 0.73199394 0.59865848 0.15601864]
```

np.random.randn():

```
[-0.1382643  0.64768854 1.52302986 -0.23415337 -0.23413696]
```

np.random.randint():

```
[21 72 83 21 87]
```

Explanation of Distributions

1. **np.random.rand()** — Continuous Uniform Distribution:
 - **Distribution:** Standard Uniform Distribution over the half-open interval $[0.0, 1.0)$.

- **Behavior:** Every floating-point number in the range $[0.0, 1.0)$ has an equal probability of being selected.

2. **np.random.randn()** — Standard Normal (Gaussian)

Distribution:

- **Distribution:** Standard Normal Distribution with a mean (μ) of 0 and a variance/standard deviation (σ) of 1 .
- **Behavior:** Generates a bell-shaped curve where roughly 68% of the values fall within $[-1, 1]$, 95% fall within $[-2, 2]$, and extreme values are rare.

3. **np.random.randint()** — Discrete Uniform Distribution:

- **Distribution:** Discrete Uniform Distribution over a specified range $[low, high)$.
- **Behavior:** Generates random integers where each integer in the range from **low** (inclusive) to **high** (exclusive) has an equal probability of being chosen.
-

Question 4: Look at the code below and answer the questions:

```
import numpy as np
arr = np.arange(12)
reshaped = arr.reshape(3, 4)
flat = reshaped.flatten()
reshaped[0, 0] = 99
print(reshaped)
print(flat)
```

What will be printed? Why does changing `reshaped[0, 0]` not affect `flat`, even though `reshape()` itself does not copy the underlying data? Also explain the difference between `np.hstack()` and `np.vstack()` with a short example.

Answer :- Python

Output of `print(reshaped)`

```
[[99  1  2  3]
```

```
 [ 4  5  6  7]
```

```
 [ 8  9 10 11]]
```

Output of `print(flat)`

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
```

Why Modifying `reshaped[0, 0]` Does Not Affect `flat`

- **`flatten()` returns a copy:** When `.flatten()` is called on an array, NumPy allocates new memory and returns a completely independent **copy** of the array's data.

- **reshape()** returns a **view**: In contrast, **.reshape()** creates a **view** of the array, sharing the same underlying data memory buffer as **arr**.

Because **flat** holds its own distinct copy of the data in memory, mutating **reshaped[0, 0]** updates the shared memory between **arr** and **reshaped**, but leaves **flat** completely untouched.

*(Note: If you want a 1D flattened view without making a copy, use **.ravel()** instead of **.flatten()**.)*

Difference Between **np.hstack()** and **np.vstack()**

- **np.hstack()** (**Horizontal Stack**): Stacks arrays **horizontally** along columns (concatenates along the second axis, **axis=1**).
- **np.vstack()** (**Vertical Stack**): Stacks arrays **vertically** along rows (concatenates along the first axis, **axis=0**).

Example Code:

Python

```
import numpy as np
```

```
a = np.array([1, 2, 3])
```

```
b = np.array([4, 5, 6])
```

```
# Stacking horizontally (side-by-side)
```

```
h_stacked = np.hstack((a, b))
```

```
print("hstack:\n", h_stacked)
```

```
# Stacking vertically (top-to-bottom)
```

```
v_stacked = np.vstack((a, b))
```

```
print("\nvstack:\n", v_stacked)
```

Output:

Plaintext

hstack:

```
[1 2 3 4 5 6]
```

vstack:

```
[[1 2 3]
```

```
[4 5 6]]
```

Question 5: What is a Pandas DataFrame, and how does it differ from a Pandas Series? Load the built-in 'tips' dataset using Seaborn, and use head(), shape, and info() to describe what each of these methods tells you about the dataset

Answer :- What is a Pandas DataFrame vs. a Pandas Series?

- **Pandas Series:** A **1-dimensional** labeled array capable of holding data of a single data type (integers, strings, floats, Python objects, etc.). It consists of two components: the data values and an associated array of data labels called the **index**.
- **Pandas DataFrame:** A **2-dimensional** tabular data structure with labeled axes (rows and columns). You can think of a DataFrame as a collection of multiple Series objects aligned together under a common index.

Feature	Pandas Series	Pandas DataFrame
Dimensions	1D	2D
Structure	Single column with an index	Rows and columns (table)
Data Types	Holds a single data type per Series	Can hold multiple data types across columns

Loading the Dataset & Method Descriptions

Python Code:

Python

```
import pandas as pd
```

```
import seaborn as sns
```

```
# Load the built-in 'tips' dataset
```

```
tips = sns.load_dataset('tips')
```

```
# 1. Preview the dataset
```

```
print("--- head() Output ---")
```

```
print(tips.head())
```

```
# 2. Check dimensions
```

```
print("\n--- shape Output ---")
```

```
print(tips.shape)
```

```
# 3. View dataset info/summary
```

```
print("\n--- info() Output ---")
```

tips.info()

What **head()**, **shape**, and **info()** Tell You

1. **head()**:

- **What it tells you:** Returns the first few rows of the DataFrame (by default, the first 5 rows).
- **Insight for tips:** Provides an immediate visual snapshot of the dataset's values, showing actual sample data for total bills, tips, customer sex, smoking status, day of the week, time of day, and party size.

2. **shape**:

- **What it tells you:** An attribute (not a function) that returns a tuple representing the dimensions of the DataFrame in the format (number_of_rows, number_of_columns).
- **Insight for tips:** Returns (244, 7), indicating that the dataset contains **244 rows (records)** and **7 columns (features)**.

3. **info()**:

- **What it tells you:** Prints a comprehensive summary of the DataFrame, including column names, non-null value counts, data types (**dtype**) for each column, index range, and memory usage.
- **Insight for tips:** Confirms that there are 244 entries with no missing values (all 244 entries are non-null across all 7 columns) and shows the specific dtypes used (e.g., **float64** for numerical values, **category** for categorical fields like **sex** or **day**, and **int64** for **size**).

Question 6: Explain the difference between `.loc[]` and `.iloc[]` in Pandas, including whether each is inclusive or exclusive of the end index. Using the Titanic dataset (`sns.load_dataset("titanic")`), write code that selects rows 0 to 5 and the columns 'sex', 'age', and 'fare' using `.loc[]`, and the first 5 rows with the 3rd and 4th columns using `.iloc[]`.

Answer :- Difference Between `.loc[]` and `.iloc[]`

- **`.loc[]` (Label-Based Indexing):**
 - **How it selects:** Selects data by **labels** (names) of rows and columns.
 - **Inclusivity:** It is **inclusive** of both the start and the end index/label (e.g., `0:5` includes both label `0` and label `5`).
- **`.iloc[]` (Integer-Position Indexing):**
 - **How it selects:** Selects data by **zero-based integer positions** (index locations), similar to standard Python lists.
 - **Inclusivity:** It is **inclusive of the start** index but **exclusive of the end** index (e.g., `0:5` includes positions `0`, `1`, `2`, `3`, `4` and excludes position `5`).

Python Code

Python

```
import pandas as pd
```

```
import seaborn as sns
```

```
# Load the Titanic dataset
```

```
df = sns.load_dataset("titanic")
```

1. Using .loc[]

Selects row labels 0 to 5 (inclusive -> 6 rows total) and specific column labels

```
loc_subset = df.loc[0:5, ['sex', 'age', 'fare']]
```

2. Using .iloc[]

Selects position rows 0 to 4 (0:5 is exclusive -> first 5 rows)

and 3rd and 4th columns (index positions 2 and 3)

```
iloc_subset = df.iloc[0:5, [2, 3]]
```

```
print("--- Using .loc[] ---")
```

```
print(loc_subset)
```

```
print("\n--- Using .iloc[] ---")
```

```
print(iloc_subset)
```

Key Points to Note

1. **`df.loc[0:5, ['sex', 'age', 'fare']]:`**

- Rows with index labels 0, 1, 2, 3, 4, 5 are selected because `.loc[ ]` includes the end boundary.

2. `df.iloc[0:5, [2, 3]]`:

- Row positions 0, 1, 2, 3, 4 (5 rows total) are selected because `.iloc[ ]` excludes index 5.
- Column index 2 represents the 3rd column, and column index 3 represents the 4th column.

Question 7: Look at the code below and answer the questions:

```
import pandas as pd
import seaborn as sns
df = sns.load_dataset("titanic")
result = df[(df['age'] > 18) & (df['survived'] == 1)]
print(result.shape)
```

What does the boolean expression inside `df[...]` select? Why are the parentheses around each condition mandatory here, and what would '&' be replaced with if you wanted rows matching EITHER condition instead of BOTH?

Answer :- **What the Boolean Expression Selects**

The boolean expression `(df['age'] > 18) & (df['survived'] == 1)` selects all passengers who:

1. Are **strictly older than 18 years** (`df['age'] > 18`), **AND**
2. **Survived** the Titanic disaster (`df['survived'] == 1`).

Why Parentheses Are Mandatory

In Python, bitwise operators like `&` (AND) and `|` (OR) have a **higher operator precedence** than comparison operators like `>` or `==`.

- **Without parentheses:** Python evaluates the expression as `df['age'] > (18 & df['survived']) == 1`. It attempts to evaluate `18 & df['survived']` first, which leads to a `TypeError` or a bitwise operation on invalid operands.
- **With parentheses:** Enclosing `(df['age'] > 18)` and `(df['survived'] == 1)` in parentheses forces Python to evaluate both comparison expressions first into two boolean Pandas Series before combining them with the bitwise `&` operator element-wise.

Logical OR Operator

To select rows matching **EITHER** condition instead of **BOTH**, replace the bitwise AND operator (`&`) with the bitwise OR operator (`|`).

Python

```
# Selects passengers who are either older than 18 OR survived (or both)
```

```
result_either = df[(df['age'] > 18) | (df['survived'] == 1)]
```

Question 8: What are the two major strategies for handling missing values in a Pandas DataFrame? Write code to: (a) drop rows where the 'age' column is missing, and (b) fill missing 'age' values with the median and missing 'embarked' values with the mode. Briefly explain when you would choose dropping over filling.

Answer :- **Two Major Strategies for Handling Missing Values**

1. **Deletion (Dropping Data):** Removing rows or columns that contain missing (NaN/None) values using `.dropna()`.
2. **Imputation (Filling Data):** Replacing missing values with estimated values (such as mean, median, mode, or a constant value) using `.fillna()`.

Python Code

Python

```
import pandas as pd
```

```
import seaborn as sns
```

```
# Load the Titanic dataset
```

```
df = sns.load_dataset("titanic")
```

```
# Strategy (a): Drop rows where the 'age' column is missing
```

```
df_dropped = df.dropna(subset=['age']).copy()
```

```
# Strategy (b): Fill missing 'age' with median and 'embarked' with mode
```

```
df_filled = df.copy()
```

```
# Fill missing 'age' with median
```

```
df_filled['age'] = df_filled['age'].fillna(df_filled['age'].median())
```

```
# Fill missing 'embarked' with mode (mode() returns a Series, so take  
index [0])
```

```
df_filled['embarked'] =  
df_filled['embarked'].fillna(df_filled['embarked'].mode()[0])
```

```
print("Original shape:", df.shape)
```

```
print("Shape after dropping missing age rows:", df_dropped.shape)
```

```
print("Missing values in filled dataset:\n", df_filled[['age',  
'embarked']].isnull().sum())
```

When to Choose Dropping Over Filling

You would choose **dropping (deletion)** over **filling (imputation)** in the following scenarios:

- **Small Proportion of Missing Data:** When the percentage of missing values is very low (e.g., $< 5\%$), dropping those rows preserves dataset integrity without significantly reducing sample size.
- **Missing Target Variable:** When missing values are in the dependent/target variable (y) that you are trying to predict.
- **High Risk of Bias:** Imputing values in sensitive columns with complex, non-random missingness patterns might introduce noise or false statistical relationships into your model.

- **Large Datasets:** When you have a vast amount of data, losing a small fraction of rows has minimal impact on analysis or model performance.

Question 9: Explain the IQR (Interquartile Range) method for detecting outliers, including how the lower and upper bounds are calculated. Write a Python program that computes Q1, Q3, and IQR for the 'fare' column of the Titanic dataset, identifies the outliers, and then caps (Winsorizes) them instead of removing them.

Answer :- Explanation of the IQR Method

The **Interquartile Range (IQR)** method is a robust, non-parametric statistical technique used to detect outliers in numerical data. It measures the spread of the middle 50% of the dataset and is unaffected by extreme values.

1. Quartiles:

- **Q1 (25th Percentile):** The value below which 25% of the data falls.
- **Q3 (75th Percentile):** The value below which 75% of the data falls.

2. IQR Calculation:

$$\text{IQR} = \text{Q3} - \text{Q1}$$

3. Outlier Boundaries:

- **Lower Bound:** $\text{Q1} - 1.5 \times \text{IQR}$
- **Upper Bound:** $\text{Q3} + 1.5 \times \text{IQR}$

Any data point falling strictly below the **Lower Bound** or above the **Upper Bound** is flagged as an **outlier**. Capping (or Winsorization) replaces extreme values exceeding these boundaries with the respective threshold value instead of dropping the records.

Python Program

Python

```
import pandas as pd
```

```
import seaborn as sns
```

```
# Load the Titanic dataset
```

```
df = sns.load_dataset("titanic")
```

```
# 1. Compute Q1, Q3, and IQR for the 'fare' column
```

```
Q1 = df['fare'].quantile(0.25)
```

```
Q3 = df['fare'].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
# 2. Calculate Lower and Upper Bounds
```

```
lower_bound = Q1 - 1.5 * IQR
```

```
upper_bound = Q3 + 1.5 * IQR
```

```
print(f"Q1 (25th percentile): {Q1:.2f}")
```

```
print(f"Q3 (75th percentile): {Q3:.2f}")
```

```
print(f"IQR: {IQR:.2f}")
```

```
print(f"Lower Bound: {lower_bound:.2f}")
```

```
print(f"Upper Bound: {upper_bound:.2f}")
```

```
# 3. Identify Outliers
```

```
outliers = df[(df['fare'] < lower_bound) | (df['fare'] > upper_bound)]  
  
print(f"\nTotal number of outliers detected: {len(outliers)}")
```

4. Cap (Winsorize) Outliers using np.clip()

```
df['fare_capped'] = df['fare'].clip(lower=lower_bound,  
upper=upper_bound)
```

```
print("\nSummary statistics before capping:")
```

```
print(df['fare'].describe())
```

```
print("\nSummary statistics after capping:")
```

```
print(df['fare_capped'].describe())
```

Program Breakdown

1. **quantile()**: Computes Q1 (0.25) and Q3 (0.75) directly from the dataset.
2. **Boundary Checks**: Determines any value less than **lower_bound** or greater than **upper_bound** as an outlier.

3. **clip()**: Caps extreme values below `lower_bound` to `lower_bound` and any values above `upper_bound` to `upper_bound`, preventing data loss while mitigating extreme skewness.

Question 10: You are preparing the Titanic dataset for a machine learning model. Write a Python program that: (a) label-encodes the binary 'sex' column, (b) one-hot encodes the 'embarked' column using `pd.get_dummies()`, and (c) standardizes the 'fare' column using Z-score scaling ($x_scaled = (x - \text{mean}) / \text{std}$). Explain why encoding and scaling are necessary before feeding data into most ML models.

Answer :- Python

```
import pandas as pd
```

```
import seaborn as sns
```

```
# Load the Titanic dataset
```

```
df = sns.load_dataset("titanic")
```

```
# --- (a) Label-encode the binary 'sex' column ---
```

```
# Map binary categories 'female' and 'male' to 0 and 1
```

```
df['sex_encoded'] = df['sex'].map({'female': 0, 'male': 1})
```

```
# --- (b) One-hot encode the 'embarked' column ---
```

```
# Use pd.get_dummies() to convert categorical variable into  
dummy/indicator variables
```

```

embarked_encoded = pd.get_dummies(df['embarked'],
prefix='embarked', dtype=int)

df = pd.concat([df, embarked_encoded], axis=1)

# --- (c) Standardize the 'fare' column using Z-score scaling ---

# Z-score formula: x_scaled = (x - mean) / std

mean_fare = df['fare'].mean()

std_fare = df['fare'].std()

df['fare_scaled'] = (df['fare'] - mean_fare) / std_fare

# Display a preview of the processed features

print(df[['sex', 'sex_encoded', 'embarked', 'embarked_C',
'embarked_Q', 'embarked_S', 'fare', 'fare_scaled']].head())

```

Output Preview:

Plaintext

	sex	sex_encoded	embarked	embarked_C	embarked_Q	embarked_S	fare	fare_scaled
0	male	1	S	0	0	1	7.2500	-0.502163

1 female	0	C	1	0	0	71.2833	
0.786404							
2 female	0	S	0	0	1	7.9250	
-0.488580							
3 female	0	S	0	0	1	53.1000	
0.420707							
4 male	1	S	0	0	1	8.0500	-0.486064

Why Encoding and Scaling Are Necessary

1. Categorical Encoding (Label Encoding & One-Hot Encoding)

- **Mathematical Compatibility:** Most machine learning algorithms (e.g., Linear Regression, Logistic Regression, Support Vector Machines, Neural Networks) perform linear algebra matrix operations under the hood. They cannot perform computations directly on raw text labels like 'male' or 'Southampton'.
- **Avoiding Implicit Ordinal Assumptions:** One-hot encoding creates separate binary (\$0/\$1) columns for non-ordinal categories so that models do not incorrectly assume an inherent numerical hierarchy or order (e.g., assuming category \$2 > 1 > 0\$).

2. Feature Scaling (Z-Score Standardization)

- **Equalizing Feature Impact:** Features with large scale magnitudes (e.g., fare ranging from \$0\$ to \$500\$) will artificially dominate optimization calculations over features with smaller numerical ranges (e.g., age or binary indicators). Scaling brings features to a shared mean of \$0\$ and standard deviation of \$1\$.

- **Gradient Descent Convergence:** Optimization algorithms (like stochastic gradient descent) converge much faster when features are on a similar scale.
- **Distance Sensitivity:** Distance-based algorithms (e.g., K -Nearest Neighbors, SVMs, K -Means Clustering) rely on spatial distances between points. Without scaling, larger scale variables inflate distance calculations significantly.